

Team Planning & Engineering Document

Driving Fatigue & Distraction Detection System

Meeting covered: Wednesday, March 26, 2025

Document date: Thursday, March 27, 2025

Phase: **Implementation** (Research phase closed)

MVP deadline: Mid-to-end May 2025

Contents

1	Meeting Summary — March 26, 2025	3
1.1	Recurring Weekly Meetings	3
1.2	Key Decisions and Information	3
1.2.1	Hardware & Camera	3
1.2.2	MediaPipe	3
1.2.3	Object Detection — YOLO and RTMDet	3
1.2.4	Head Pose Estimation	3
1.2.5	Data and Prior Work	3
2	Repository Structure	3
2.1	Overview	3
2.2	Folder Layout	3
3	Git Workflow	4
3.1	Branch Strategy	4
3.2	Branch Naming Conventions	5
3.3	Workflow Rules	5
4	Coding Standards & Conventions	5
4.1	Language and Style	5
4.2	Docstrings	5
4.3	Configuration — No Hardcoded Values	6
4.4	Logging — No <code>print()</code> in Production Code (until the end)	6
4.5	Notebooks	6
5	Module-by-Module Implementation Plan	6
5.1	Module 1 — Face Mesh & Head Pose Ashref	6
5.1.1	Goal	6
5.1.2	Approach	6
5.1.3	Done When	6
5.2	Module 2 — Geometric Fatigue Detection Ashref	6
5.2.1	Goal	7
5.2.2	Approach	7
5.2.3	Done When	7
5.3	Module 3 — Gaze Estimation Ashref	7
5.3.1	Goal	7
5.3.2	Approach	7
5.3.3	Done When	7
5.4	Module 4 — Deep Learning Fatigue Hamza	7
5.4.1	Goal	7
5.4.2	Approach	7
5.4.3	Go / No-Go Criteria (evaluated at end of M2)	8

5.5	Module 5 — Compliance Detection	Imen & Yacine	8
5.5.1	Goal		8
5.5.2	Approach		8
5.5.3	Done When		8
5.6	Module 6 — Integration Pipeline	All	8
5.6.1	Goal		8
5.6.2	Approach		8
6	Weekly Milestones		9
6.1	Gantt Chart		11
7	Open Items & Blockers		11
8	Communication Protocol		11

Meeting Summary — March 26, 2025

Recurring Weekly Meetings

Every Thursday, approximately 8:00 PM (T.B.D.), Google Meet. All team members must attend. Before each session, every member prepares a brief status update on their task and lists any blockers or questions for the supervisor.

Key Decisions and Information

Hardware & Camera

The edge device is confirmed: **Raspberry Pi 4**. The camera will be visible-light only for now. An IR camera was ordered but has not arrived. IR support is a known limitation to document — it does not block any work.

MediaPipe

The supervisor confirmed MediaPipe for face landmark detection. He mentioned downloading the `.task` file for the Face Landmarker locally. This is informational — we may use the pip-installed version during development. The model variant (Lite/Full/Heavy) must be settled before building the pipeline.

Object Detection — YOLO and RTMDet

YOLO is approved for the development phase. The supervisor's key point: *get the data and pipeline right first — model swapping is trivial once the pipeline is solid*. He also mentioned **RTMDet** positively in the context of object detection. **Imen & Yacine** are assigned to research this.

Head Pose Estimation

Flagged by the supervisor as a core challenge. It is not optional — both the distraction branch (gaze depends on orientation) and the drowsiness branch (EAR/MAR degrade on off-angle faces) require it. A working solution must be part of the MVP.

Data and Prior Work

The supervisor will send us a dataset and prior work from his end. **Neither has arrived yet.** **Ashref** to follow up by this week.

Repository Structure

Overview

The repository lives on **GitHub**. All members have push access. The structure separates source code cleanly by module, following our parallel team split, so members do not step on each other's work.

Folder Layout

```
fatigue-detection/
|-- README.md
|-- requirements.txt           # pinned pip dependencies
|-- requirements-dev.txt      # dev tools: pytest, black, flake8
|-- config/
|   |-- config.yaml          # all thresholds, paths, flags (no
|       hardcoding)
|-- data/                    # .gitignored entirely
|   |-- raw/
|   |-- processed/
|   '-- splits/
```

```

|-- notebooks/                # Colab-ready .ipynb files for training
|   |-- ashref/
|   |-- hamza/
|   '-- imen_yacine/
|-- src/
|   |-- __init__.py
|   |-- face_mesh/           # Ashref
|   |   |-- __init__.py
|   |   |-- detector.py      # MediaPipe wrapper, landmark extraction
|   |   '-- head_pose.py     # solvePnP or MediaPipe built-in
|   |-- fatigue/
|   |   |-- __init__.py
|   |   |-- geometric.py     # EAR, MAR, PERCLOS (Ashref)
|   |   |-- gaze.py          # gaze estimation (Ashref)
|   |   '-- dl_model.py      # CNN + GRU (Hamza)
|   |-- compliance/         # Imen & Yacine
|   |   |-- __init__.py
|   |   |-- detector.py      # YOLO / RTMDet wrapper
|   |   '-- smoking.py       # landmark-based smoking detection
|   |-- pipeline/
|   |   |-- __init__.py
|   |   '-- runner.py        # main loop, integrates all modules
|   '-- utils/
|       |-- __init__.py
|       |-- logger.py
|       |-- video_io.py      # webcam / file abstraction
|       '-- alerts.py        # alert trigger logic
|-- models/                  # .gitignored (weights, .tflite files)
|-- tests/
|   |-- test_geometric.py
|   |-- test_detector.py
|   '-- test_pipeline.py
|-- scripts/
|   |-- run_demo.py          # entry point for the live Pi demo
|   |-- export_tflite.py     # convert models for edge
|   '-- evaluate.py
 '-- docs/
    '-- reports/

```

Never commit: dataset files, model weights (.pt, .tflite, .task), or any file over 50 MB. Add them to .gitignore from day one. Use a shared drive link for weights distribution within the team.

Git Workflow

Branch Strategy

We follow a simplified **Git Flow** with three branch types:

Branch	Purpose
main	Stable, demo-ready code only. Only merged into at milestones. Protected — no direct pushes.
dev	Integration branch. All feature branches merge here. Always runnable.
feature/ <i>name</i>	Individual work. One branch per feature or module. Short-lived.
fix/ <i>name</i>	Bug fixes on dev.

Branch Naming Conventions

```
feature/face-mesh-baseline
feature/ear-mar-computation
feature/head-pose-solvepnp
feature/dl-fatigue-cnn-gru
feature/compliance-yolo-baseline
feature/tflite-export
fix/ear-threshold-calibration
```

Workflow Rules

1. **Never push directly to main or dev.** Always work on a **feature/** branch.
2. **Open a Pull Request** to merge into dev. At least one other team member must review and approve before merge.
3. **Commit messages** follow the format: `type: short description`
Types: feat, fix, refactor, data, docs, test
Example: feat: add EAR computation with landmark indices
4. **Pull from dev before opening a PR** to resolve conflicts on your side, not the reviewer's.
5. **Milestones = merge to main.** Must be reviewed by other member(s), at the end of each milestone.
6. **One feature per branch.** Don't bundle unrelated changes.

Coding Standards & Conventions

Language and Style

All code is **Python 3.10+**. Shell scripts for automation are acceptable but must be documented.

Docstrings

Every function and class must have a docstring in **Google style**:

```
def compute_ear(landmarks: np.ndarray, eye_indices: list[int]) ->
float:
    """Compute Eye Aspect Ratio (EAR) from facial landmarks.

    Args:
        landmarks: Array of shape (468, 3) from MediaPipe Face Mesh.
        eye_indices: List of 6 landmark indices for one eye.

    Returns:
        EAR value as a float. Values below 0.2 typically indicate
        closure.
    """
```

Configuration — No Hardcoded Values

All thresholds, paths, and flags go in `config/config.yaml`. No magic numbers in source files.

```
# WRONG
if ear < 0.21:
    trigger_alert()

# CORRECT (load config at startup)
if ear < cfg["fatigue"]["ear_threshold"]:
    trigger_alert()
```

Logging — No `print()` in Production Code (until the end)

Use the `utils/logger.py` module. Reserve `print()` for notebooks only.

```
from utils.logger import get_logger
log = get_logger(__name__)

log.info("Pipeline started.")
log.warning("Low EAR detected: %.3f", ear)
log.error("Camera feed lost.")
```

Notebooks

Notebooks live in `notebooks/{name}/`. They are for exploration and Colab training only — **no notebook code is used in the pipeline directly**. Once something works in a notebook, it gets ported to `src/` as clean Python.

Module-by-Module Implementation Plan

This section details what each module must do, the approach to use, and the output it must produce to be considered complete.

Module 1 — Face Mesh & Head Pose

Ashref

Goal

Produce, for every frame: (1) a 468-landmark face mesh, (2) 3D head orientation (pitch, yaw, roll), (3) normalised landmark positions corrected for head angle.

Approach

1. Wrap **MediaPipe Face Mesh** (decide Lite vs Full based on Pi performance benchmark).
2. Implement head pose estimation using **solvePnP** with a 3D face model and selected stable landmarks (nose tip, chin, eye corners, mouth corners). MediaPipe's built-in `facial_transformation_matrix` is an alternative if solvePnP is too slow on Pi.
3. Output pitch, yaw, roll in degrees.
4. Add a validity flag: if face is not detected or head angle exceeds ± 45 yaw / ± 30 pitch, downstream modules must be notified.

Done When

Processes a test video at ≥ 15 FPS on laptop, outputs stable pitch/yaw/roll visualised on frame, handles no-face frames gracefully without crashing.

Module 2 — Geometric Fatigue Detection

Ashref

Goal

Compute EAR, MAR, and PERCLOS from landmarks. Emit drowsiness and yawning alerts with configurable thresholds.

Approach

1. **EAR** (Eye Aspect Ratio): standard formula using 6 landmarks per eye. Average both eyes.
2. **MAR** (Mouth Aspect Ratio): analogous formula for mouth opening (yawn detection).
3. **PERCLOS**: rolling time window (default 60 s), percentage of frames where EAR < threshold. Alert at PERCLOS > 0.15.
4. Correct landmark positions using head pose output from Module 1 (do not compute EAR on a heavily rotated face without correction).
5. All thresholds in `config.yaml`.

Done When

Correctly classifies eye-closed vs eye-open on MRL Eye dataset test split. PERCLOS triggers alert on a test drowsy video from NTHU-DDD.

Module 3 — Gaze Estimation

Ashref

Goal

Estimate where the driver is looking (left, right, forward, down) to detect distraction.

Approach

1. Use MediaPipe iris landmarks (landmarks 468–477) for iris centre estimation.
2. Compute gaze direction relative to eye bounding box.
3. Map to discrete zones: forward, left, right, down. Distraction alert if driver looks away for > 2 s.
4. This module is **dependent on Module 1** (head pose) — begin only once head pose is stable.

Done When

Correctly classifies gaze direction on a set of labelled test frames. Alert triggers correctly on a video of a driver looking away.

Module 4 — Deep Learning Fatigue

Hamza

Conditional module. Hamza will attempt the DL approach. If the model achieves acceptable accuracy *and* runs within the edge constraints of the Pi, he continues developing it. If either condition is not met after a fair attempt (M3), **Hamza joins Ashref** on the geometric fatigue and gaze pipeline instead. The DL branch is a bonus, not a hard requirement for the MVP.

Goal

A learned fatigue classifier that goes beyond geometric thresholds — detects subtle drowsiness states from facial appearance over time.

Approach

1. **Backbone:** MobileNetV3-Small (pretrained ImageNet). Input: cropped eye region from Module 1.
2. **Temporal model:** GRU on top of CNN frame features. Sequence length: 20–30 frames.
3. **Training:** On Colab (GPU). Primary dataset: NTHU-DDD. Secondary: UTA-RLDD. Fine-tune on supervisor data when received.

4. **Export:** TFLite INT8 quantised for Pi. Target: <5 MB model size.
5. **Notebooks:** All training in `notebooks/hamza/`. Clean inference class in `src/fatigue/dl_model.py`.

Go / No-Go Criteria (evaluated at end of M2)

- **Accuracy:** $\geq 80\%$ on NTHU-DDD test set.
- **Edge viability:** TFLite INT8 model runs at ≥ 10 FPS on Pi with <5 MB size.
- If both conditions are met $\rightarrow$ continue to M3 and integration.
- If either condition is not met $\rightarrow$ Hamza pivots to support **Ashref** on Modules 2 & 3.

Module 5 — Compliance Detection

Imen & Yacine

Goal

Detect seatbelt non-compliance, phone use, and smoking in real time.

Approach

Seatbelt and Phone — Object Detection:

1. Begin with **YOLOv10n INT8** (Apache 2.0, NMS-free, fast). Research **RTMDet** as alternative.
2. RTMDet research task: understand architecture, compare latency and mAP vs YOLOv10n on a sample, decide which to use. Produce a written comparison before next Thursday.
3. Train on: NCAI seatbelt dataset (760 in-vehicle images) + Roboflow merged for phone. Supplement with supervisor data when received.
4. Export to TFLite INT8.

Smoking — Landmark-Based:

1. Object detection is unreliable for cigarettes at distance. Use hand-to-mouth proximity via MediaPipe Hands + Face Mesh landmarks.
2. Alert if hand landmark is within N pixels of mouth landmark for > 1 s.

Done When

Seatbelt and phone detection achieves $\geq 75\%$ mAP on held-out test set. Smoking detection correctly fires on a test video with visible hand-to-mouth gesture.

Module 6 — Integration Pipeline

All

Goal

A single `scripts/run_demo.py` entry point that runs all modules in parallel on a live camera feed or video file, overlays results on the frame, and triggers alerts.

Approach

1. **Frame loop:** capture frame $\rightarrow$ face mesh $\rightarrow$ head pose $\rightarrow$ parallel: [geometric fatigue, DL fatigue, gaze, compliance] $\rightarrow$ alert logic $\rightarrow$ annotated frame output.
2. Use Python **threading** for module parallelism on Pi. Avoid **multiprocessing** (Pi RAM is limited).
3. Annotated frame: draw landmarks, display EAR/MAR/PERCLOS values, show alert banners.
4. Alert system: configurable per alert type (audio beep, on-screen banner, log entry).

Weekly Milestones

The project has approximately **7–8 weeks** remaining.

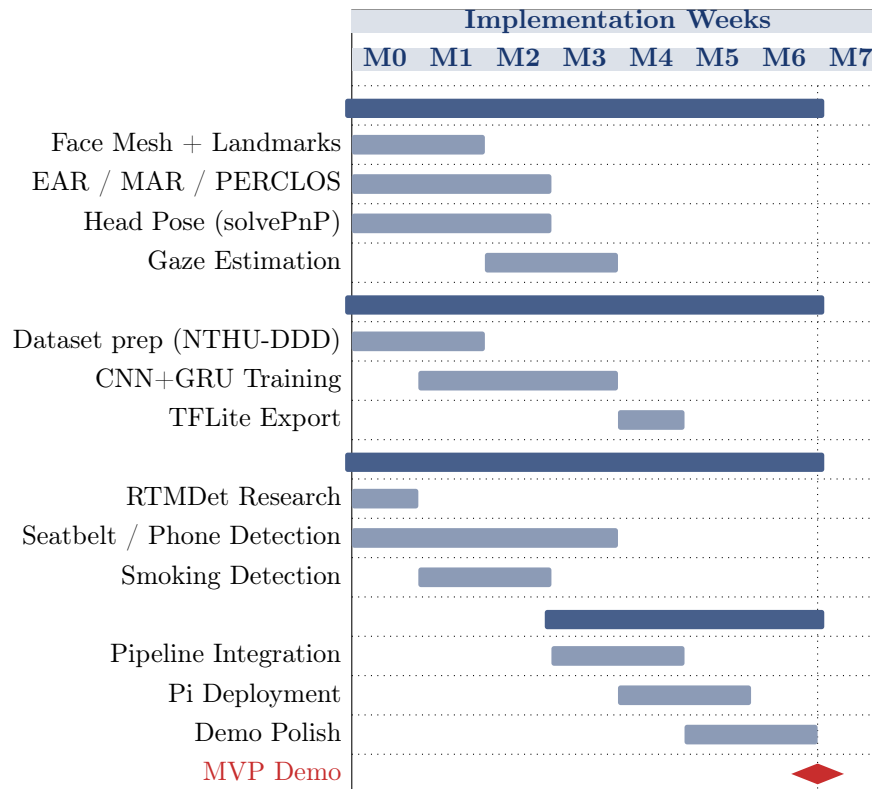
These milestones are estimates, not a fixed contract. The schedule will shift as we receive supervisor data, get feedback on Thursday meetings, and discover implementation challenges. What matters is the *order* (setup → baselines → modules → integration → edge → demo) and the *direction* — not the exact dates. Adjust openly at each Thursday meeting rather than silently slipping.

Milestone Rules

- A milestone is **code that runs**, not a written plan or a notebook that half-works.
- Every milestone completion = a merge from **dev** to **main**.
- If you are blocked, say so in the team chat **immediately** — not on Thursday.

Week	Dates	Milestone
M0	Mar 28 – Apr 3	Setup week. Repo structure created, <code>.gitignore</code> set up, <code>requirements.txt</code> pinned, <code>config.yaml</code> skeleton written. Each member creates their branch. Follow up on supervisor data. RTMDet research begins. MediaPipe model variant decided. Ashref: MediaPipe face mesh running, landmarks drawn on live webcam feed. Hamza: Colab environment set up, NTHU-DDD loaded and explored. Imen & Yacine: YOLOv10n running on sample images, RTMDet research started.
M1	Apr 4 – Apr 10	Core baselines. Ashref: EAR and MAR computing correctly from landmarks. Head pose pitch/yaw/roll displayed on frame. Hamza: MobileNetV3-Small baseline trained (even if low accuracy), training loop clean and reproducible. Imen & Yacine: YOLOv10n trained on seatbelt dataset (first pass). RTMDet comparison written and shared.
M2	Apr 11 – Apr 17	Modules functional. Ashref: PERCLOS rolling window implemented. Gaze estimation first pass (iris landmarks). Head pose correction applied to EAR/MAR. Hamza: CNN+GRU architecture implemented. Training on NTHU-DDD with sequence data. Go/No-Go decision at end of this week — if accuracy and Pi FPS targets are not realistic, Hamza pivots to support Ashref. Imen & Yacine: Phone detection baseline. Smoking hand-to-mouth logic implemented and tested on video.
M3	Apr 18 – Apr 24	Modules validated. All modules meet their individual “Done When” criteria from Section 5. Supervisor data integrated if received. Modules pass unit tests in <code>tests/</code>.
M4	Apr 25 – May 1	Integration. All: <code>pipeline/runner.py</code> integrates all modules. <code>run_demo.py</code> works end-to-end on a laptop with webcam. Annotated frame output correct. Basic alert system working.
M5	May 2 – May 8	Edge deployment. All models exported to TFLite INT8. Pipeline tested on Raspberry Pi 4. Frame rate measured. Bottlenecks identified and addressed. Target: ≥ 10 FPS end-to-end on Pi.
M6	May 9 – May 15	Demo-ready MVP. Full pipeline running stably on Pi with live camera feed. All alerts functional. Clean demo script. Edge cases handled (no face detected, poor lighting). System can run for 30+ minutes without crash.
M7	May 16 – May 22	Buffer + polish. Bug fixes, demo preparation, documentation cleanup. Academic report finalised. Supervisor sign-off. Project submitted.

Gantt Chart



Open Items & Blockers

- Supervisor data [BLOCKER]** Not received. **Ashref** follows up by this week. Do not finalise dataset splits until received.
- Supervisor prior work [BLOCKER]** Not received. May affect DL architecture decisions. **Hamza** keeps training pipeline flexible.
- RTMDet research [Imen & Yacine]** Research RTMDet: architecture, speed vs accuracy vs YOLOv10n, suitability for in-vehicle edge deployment. Produce a 1–2 page written comparison before next Thursday (April 3).
- Head pose implementation plan [Ashref]** Research solvePnP vs MediaPipe built-in. Prototype both and benchmark on Pi. Decision made by end of M0.
- MediaPipe variant [Ashref]** Decide Lite/Full/Heavy by benchmarking on Pi or estimating from published latency numbers. Add decision to `config.yaml` comment.
- MVP scope with supervisor [All]** Raise at next Thursday meeting: exactly what is required for academic validation?

Communication Protocol

- Day-to-day:** Team chat (WhatsApp / Telegram). Blockers posted immediately, not held for Thursday.
- Weekly Thursday meeting:** Each member prepares a 2-minute verbal update: what was done, what is next, any blockers.
- PR reviews:** Reviewed within 24 hours of opening. Don't let PRs sit for days.

Document maintained by Ashref — update after every Thursday meeting.
Last updated: March 27, 2025.